

M07 Learner Book

PostgreSQL + Data Modeling | Release Candidate 1 | 13 controlled lessons

M07-L01 - Install/verify PostgreSQL and connect safely

Learning objective

Verify the server/client/database you are actually using.

Why this matters in Data Engineering

SQL behavior and diagnostics depend on the real engine and connection. A GUI is only a client; the database server remains the source of truth.

Understand

- PostgreSQL server stores data; psql and pgAdmin are clients.
- `SELECT version()`, `current_database()` and `current_user` verify the connection context.
- Use a dedicated course database rather than experimenting in an unrelated database.
- Record installation/version evidence; do not blindly reinstall a working server.

Try / inspect

```
psql --version
psql -d stage07_de
SELECT version();
SELECT current_database(), current_user;
```

Common failure

Confusing a GUI connection profile with the database itself or running setup in the wrong database.

Your turn - independent proof

Create/verify stage07_de and save connection/version evidence plus one query result.

Move on when

You can name the server version, database and current role from query evidence.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L02 - Tables, data types and explicit DDL

Learning objective

Define columns/types/defaults intentionally instead of relying on permissive text storage.

Why this matters in Data Engineering

Data contracts start at the table boundary. Types and nullability prevent impossible states from silently spreading.

Understand

- Choose types from business meaning and realistic bounds.
- NOT NULL means absence is not permitted; defaults are inserted values, not documentation.
- Use NUMERIC/DECIMAL for exact financial-style values when appropriate.
- DDL belongs in version-controlled setup scripts.

Try / inspect

```
CREATE TABLE suppliers (  
  supplier_id text PRIMARY KEY,  
  supplier_name text NOT NULL,  
  active boolean NOT NULL DEFAULT true,  
  rating int CHECK (rating BETWEEN 1 AND 5)  
);
```

Common failure

Using text for every field and trusting application code to enforce all validity rules.

Your turn - independent proof

Design a new deliveries table with justified types, nullability and defaults; run it in a disposable schema.

Move on when

You can explain each column type/constraint from the business contract.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L03 - Primary keys, foreign keys and constraints

Learning objective

Protect identity and relationships with database-enforced rules.

Why this matters in Data Engineering

A pipeline that loads orphan or duplicate business entities can produce plausible but untrustworthy analytics.

Understand

- Primary keys define row identity.
- Foreign keys protect referenced relationships.
- CHECK enforces allowed row-level predicates.
- Constraint failures are useful evidence; do not remove constraints just to make a load green.

Try / inspect

```
ALTER TABLE orders ADD CONSTRAINT fk_customer
FOREIGN KEY (customer_id) REFERENCES customers(customer_id);

CHECK (quantity > 0)
```

Common failure

Treating a constraint error as a nuisance and weakening the contract instead of fixing the data/logic.

Your turn - independent proof

Trigger controlled PK, FK, NOT NULL and CHECK failures and save each error without removing the constraints.

Move on when

You can predict which invalid row should fail and why.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L04 - Transactions, DML and UPSERT

Learning objective

Make related writes atomic and design rerunnable insert/update behavior.

Why this matters in Data Engineering

Partial writes and duplicate retries are routine failure modes in data pipelines.

Understand

- BEGIN/COMMIT define an atomic transaction boundary.
- ROLLBACK returns the transaction to the prior committed state.
- ON CONFLICT can implement controlled idempotent upsert logic.
- The conflict key and update rule must match business semantics.

Try / inspect

```
BEGIN;  
UPDATE customers SET region='East' WHERE customer_id='C002';  
ROLLBACK;  
  
INSERT ... ON CONFLICT (customer_id) DO UPDATE SET ...;
```

Common failure

Using UPSERT without a stable key or updating every column blindly on conflict.

Your turn - independent proof

Prove rollback, then implement one rerunnable upsert and rerun it twice without duplicate rows.

Move on when

You can explain what commits atomically and why a rerun is safe.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L05 - Indexes and EXPLAIN evidence

Learning objective

Use query plans to reason about access paths without demanding a specific plan.

Why this matters in Data Engineering

Indexes can speed selective access but add write/storage cost; tiny tables may rationally scan sequentially.

Understand

- EXPLAIN shows the planned operators; EXPLAIN ANALYZE also executes.
- A Seq Scan is not automatically bad.
- Indexes should follow observed query patterns and selectivity.
- Compare output correctness before and after tuning.

Try / inspect

```
EXPLAIN SELECT * FROM orders WHERE customer_id='C001';  
CREATE INDEX idx_orders_customer ON orders(customer_id);  
EXPLAIN SELECT * FROM orders WHERE customer_id='C001';
```

Common failure

Creating indexes on every column or claiming a plan improved just because an index exists.

Your turn - independent proof

Compare one plan before/after an index and report only what actually changes in your environment.

Move on when

You can identify the main scan/join operator and explain why the chosen plan is plausible.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L06 - Operational vs analytical modeling and normalization

Learning objective

Separate write-oriented operational design from analysis-oriented warehouse design.

Why this matters in Data Engineering

Copying descriptive attributes into every event row creates update anomalies and unclear ownership.

Understand

- Operational systems often normalize to reduce inconsistent repeated attributes.
- Analytical systems intentionally denormalize through facts/dimensions for consistent slicing.
- The same business concept can have different physical models by workload.
- Start with grain and update semantics, not diagram aesthetics.

Try / inspect

```
Operational: customers <- orders <- order_items  
Analytical: dim_customer + dim_product + dim_date -> fact_sales
```

Common failure

Calling every wide table a star schema or every normalized table unsuitable for analytics.

Your turn - independent proof

Given a mixed operational table, identify repeating attributes and propose normalized source plus analytical target models.

Move on when

You can justify where an attribute belongs based on identity, update behavior and query use.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L07 - Facts, dimensions and star-schema grain

Learning objective

Declare fact grain and dimension meaning before building analytical joins.

Why this matters in Data Engineering

Wrong grain causes double counting even when every query is syntactically correct.

Understand

- Fact grain states what one row represents.
- Dimensions provide descriptive context at stable keys.
- Measures must be valid at the fact grain.
- Join paths should not multiply the fact unexpectedly.

Try / inspect

```
fact_sales grain = one completed order line
Dimensions = customer, product, date
Measures = quantity, revenue
```

Common failure

Joining a many-valued descriptive table directly to a fact and silently multiplying measures.

Your turn - independent proof

State fact_sales grain, calculate revenue by region/category, and reconcile totals back to the fact.

Move on when

You can explain every key in the fact and prove aggregation totals reconcile.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L08 - Surrogate keys and referential integrity

Learning objective

Use warehouse-controlled identity while preserving source business keys.

Why this matters in Data Engineering

Source identifiers can be reused, changed or originate from multiple systems; warehouse keys separate internal identity from source representation.

Understand

- Business/natural key comes from the source domain.
- Surrogate key is generated/managed by the analytical model.
- Facts normally reference dimension surrogate keys.
- Unknown/late dimensions require an explicit policy, not orphan facts.

Try / inspect

```
dim_customer(customer_key bigint, customer_id text, ...)  
fact_sales(..., customer_key bigint REFERENCES dim_customer)
```

Common failure

Replacing the source business key entirely and losing the ability to trace back to the source.

Your turn - independent proof

Create a surrogate-key dimension, load facts through a lookup, and prove an invalid dimension key cannot enter the fact.

Move on when

You can trace from fact surrogate key back to the source business key and explain the policy for missing dimensions.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L09 - Slowly Changing Dimensions and history

Learning objective

Preserve selected attribute history without ambiguous overlapping current rows.

Why this matters in Data Engineering

Analytics may need both current context and historical context at the time an event occurred.

Understand

- SCD1 overwrites selected attributes; SCD2 preserves versions.
- SCD2 needs business key, surrogate key, effective dates and current/version rule.
- Exactly one current row per business key is a common invariant.
- Date ranges must not overlap for the same business key.

Try / inspect

```
customer_id=C003  
old: North, 2026-01-01..2026-08-31, current=false  
new: East, 2026-09-01..9999-12-31, current=true
```

Common failure

Updating the old row in place and destroying history or creating two simultaneous current versions.

Your turn - independent proof

Run the SCD2 lab with a new effective date and write validation queries for one-current-row and non-overlap.

Move on when

You can show prior/current versions and explain how a fact chooses the correct version.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L10 - Build and validate a small analytical PostgreSQL database

Learning objective

Integrate model, load and validation into one reproducible database build.

Why this matters in Data Engineering

A schema diagram is not enough; the loaded database must satisfy row counts, totals, keys and rerun behavior.

Understand

- Build scripts should be rerunnable or clearly reset controlled objects.
- Validate row counts and key uniqueness before business totals.
- Reconcile regional/category totals to the whole fact.
- Save validation SQL with the project.

Try / inspect

```
SELECT count(*) FROM fact_sales;  
SELECT sum(quantity), sum(revenue) FROM fact_sales;  
SELECT order_id, line_no, count(*) FROM fact_sales GROUP BY 1,2 HAVING count(*)>1;
```

Common failure

Declaring success because the final chart/query looks plausible without independent reconciliation.

Your turn - independent proof

Run the provided build and prove 12 fact rows, qty 30, revenue 14130.00, no cancelled facts, no orphan keys and no duplicate order-line grain.

Move on when

All required controls reconcile and can be rerun from version-controlled SQL.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L11 - Bulk loading with COPY and controlled staging

Learning objective

Load file data efficiently into a staging boundary before transforming into governed tables.

Why this matters in Data Engineering

Real pipelines load batches. A staging table isolates raw/file-shaped data from business-constrained serving models.

Understand

- COPY is server-side; \copy is client-side psql behavior.
- Load into a staging table with source metadata rather than bypassing validation.
- Validate row count/header/type expectations before promoting data.
- Keep raw file identity/checksum/run metadata where practical.

Try / inspect

```
\copy stg_orders FROM 'orders.csv' CSV HEADER
SELECT count(*) FROM stg_orders;
INSERT INTO orders (...) SELECT ... FROM stg_orders WHERE ...;
```

Common failure

Bulk loading directly into a final constrained table and then disabling constraints when dirty rows fail.

Your turn - independent proof

Load a fresh CSV into a staging table, profile it, reject one bad row and promote only valid rows.

Move on when

You can explain the staging-to-target boundary and reconcile file rows to promoted/rejected outcomes.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L12 - Schemas, roles and least-privilege boundaries

Learning objective

Separate namespaces/permissions for ingestion, transformation and read-only consumption.

Why this matters in Data Engineering

Production systems should not give every process owner-level access to every object.

Understand

- Schemas organize object namespaces and ownership boundaries.
- Roles can own objects or receive privileges.
- Ingestion should not need to DROP unrelated schemas; analysts often need read-only access.
- Secrets/credentials belong outside committed SQL files.

Try / inspect

```
CREATE SCHEMA warehouse;  
GRANT USAGE ON SCHEMA warehouse TO analyst_ro;  
GRANT SELECT ON ALL TABLES IN SCHEMA warehouse TO analyst_ro;
```

Common failure

Using the database superuser for every application/pipeline because permissions are inconvenient.

Your turn - independent proof

Design three roles (owner/loader/reader) and write the minimum grants for a small warehouse schema.

Move on when

You can state what each role can and cannot do and why.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M07-L13 - Schema change, backup/restore and handoff validation

Learning objective

Treat database structure changes as versioned, recoverable engineering changes.

Why this matters in Data Engineering

A database that cannot be recreated or safely changed is fragile even if current queries work.

Understand

- Version DDL migrations alongside code.
- Make destructive changes explicit and backed by rollback/backup plans.
- pg_dump/pg_restore concepts provide logical backup/recovery; learner execution depends on local setup.
- Handoff should include build order, dependency/version notes and validation queries.

Try / inspect

```
pg_dump -Fc stage07_de > stage07_de.dump
# version a migration before applying
# rerun validation after change
```

Common failure

Editing production-like schema manually with no migration record or post-change reconciliation.

Your turn - independent proof

Create one safe additive migration, document rollback/restore approach, rerun core validation and write a clean-machine handoff note.

Move on when

Another learner can recreate the schema/data path and verify post-change correctness from saved materials.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.